

NR4SD⁺ / Booth, 2 PPs

DP8 (8×4) — NR4SD⁺ low digit, Booth top digit, two per nibble

Step 1 — the dot product

$$\sum_{i=0}^7 A_i B_i$$

Step 2 — recode B into radix-4 digits and exchange the sums

Each B_i is a 4-bit value read as signed, plus a carry in c_i . The low pair is recoded NR4SD⁺ and pushes a carry c'_i into the top pair, which is read as a Booth digit:

$$s_i = 2 B_i^1 + B_i^0 + c_i \quad c'_i = \lceil s_i \geq 3 \rceil \quad d_i^0 = s_i - 4 c'_i \in \{-1, 0, 1, 2\} \quad d_i^1 = -2 B_i^3 + B_i^2 + c'_i \in \{-2, \dots, 2\}$$

$$\sum_{i=0}^7 A_i B_i = \sum_{i=0}^7 \left[\sum_{j=0}^1 4^j d_i^j \right] A_i = \sum_{j=0}^1 4^j \sum_{i=0}^7 d_i^j A_i$$

Two NR4SD⁺ digits span $[-5, 10]$ and miss $-6, -7, -8$; the Booth top digit brings the -2 , so two digits span $[-9, 10]$ and cover a signed nibble plus carry in. The carry in c_i is 0 for a whole 4-bit operand and the top bit of the nibble below when B_i is a slice of a wider B (Step 5).

Step 3 — the partial products

$$\sum_{i=0}^7 A_i B_i = \sum_{i=0}^7 \underbrace{\left[d_i^0 A_i \right]}_{R_0(B_i^1, B_i^0, c_i)} + 4 \sum_{i=0}^7 \underbrace{\left[d_i^1 A_i \right]}_{R_1(B_i^3, B_i^2, c'_i)}$$

$R_0(B_i^1, B_i^0, c_i)$ depends on the low pair and the carry in and takes one of four values; $R_1(B_i^3, B_i^2, c'_i)$ is the Booth table with c'_i as the bit below:

B_i^1	B_i^0	c_i	c'_i	d_i^0	R_0	B_i^3	B_i^2	c'_i	d_i^1	R_1
0	0	0	0	0	0	0	0	0	0	0
0	0	1	0	+1	A_i	0	0	1	+1	A_i
0	1	0	0	+1	A_i	0	1	0	+1	A_i
0	1	1	0	+2	$2A_i$	0	1	1	+2	$2A_i$
1	0	0	0	+2	$2A_i$	1	0	0	-2	$-2A_i$
1	0	1	1	-1	$-A_i$	1	0	1	-1	$-A_i$
1	1	0	1	-1	$-A_i$	1	1	0	-1	$-A_i$
1	1	1	1	0	0	1	1	1	0	0

Step 4 — split the work

The encoding is a function of B alone, so it is done in the dispatcher, which sends the digits d_i^j — the control signals of the selection multiplexers — to the PE; only the selection of the multiple stays inside:

$$\underbrace{\{B_i^1, B_i^0, c_i\} \longrightarrow d_i^0}_{\text{dispatcher: encoding}} \quad \underbrace{\{B_i^3, B_i^2, c'_i\} \longrightarrow d_i^1}_{\text{PE: selection}} \longrightarrow \underbrace{\sum_{i=0}^7 d_i^0 A_i + 4 \sum_{i=0}^7 d_i^1 A_i}_{\text{PE: selection}}$$

d_i^0 travels as its 2-bit code $(B_i^0 \oplus c_i, B_i^1 \oplus (B_i^0 \wedge c_i))$, with $00 \rightarrow 0, 01 \rightarrow +1, 10 \rightarrow +2, 11 \rightarrow -1$, which the low cell uses directly; d_i^1 travels as the Booth window $\{B_i^3, B_i^2, c'_i\}$ with $c'_i = B_i^1 \wedge (B_i^0 \vee c_i)$, which the top cell decodes: 5 control bits per lane.

Step 5 — slices of a wider B

A lower nibble L read as signed is short by $16 b_3$; the carry in hands $+b_3$ to the upper nibble H in the DP8 above, and the $\times 16$ of the tree cancels the two, element by element, whatever the grouping of the DP8s:

$$16(H + b_3)A + (L - 16b_3)A = (16H + L)A$$

B is the narrow operand: 2 digits $\times$ 8 lanes = 16 selections of 10 bits per DP8, the low one over $\{0, A_i, 2A_i, -A_i\}$ (2 control bits), the top one over $\{0, \pm A_i, \pm 2A_i\}$ (3 control bits).